

“The essence of training is to allow error without consequence.”

Orson Scott Card, Ender's Game



CMPUT 628

Deep RL

Marlos C. Machado

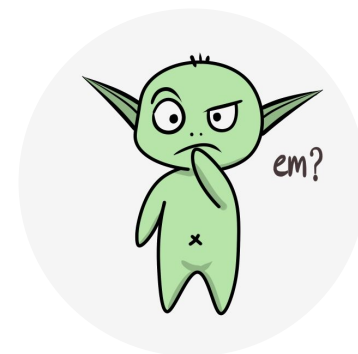
<https://robert-simons-pevb.squarespace.com/enders-game-ships>

Class 19/ 25

Reminders & Notes

- Assignment 4 is due on March 28 — Later than what's in the syllabus
 - I adjusted the performance criteria for each algorithm
- Midterm is Wednesday
 - Bring a pen (ideally pencil so you don't make a mess). Get ready to write!
 - Your answers need to be legible. I can't mark what I don't understand.
 - Sit apart from each other.
 - Bring your OneCard or a different piece of ID.
 - If you are not taking this class for credit, don't come :-D
- Seminars start next week
 - Ryan Cooper and Shway Wang; Matthew Vandergrift and Steven Tang
- This is my last actual lecture, so no more reminders & notes
 - I will be sharing a feedback form with you after the midterm

Please, interrupt me at any time!



Last Class: MuZero [Schrittwieser et al., 2020]

- **Learns a Model for Planning**

Trains a latent dynamics model that predicts *only* what is needed for planning—state representations, rewards, values, and policies—without reconstructing observations

- **Uses MCTS for Decision-Making**

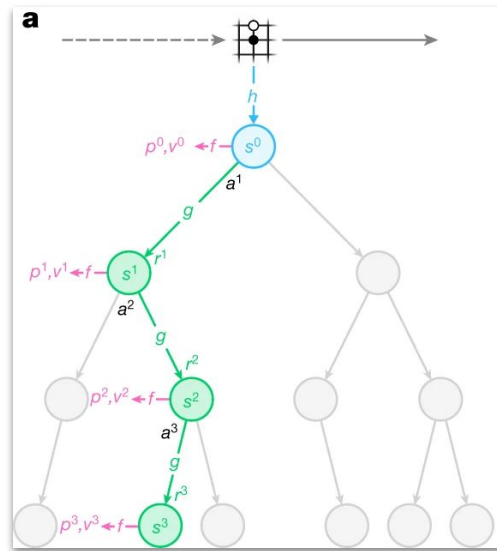
At each timestep, MCTS expands possible future sequences to refine action selection

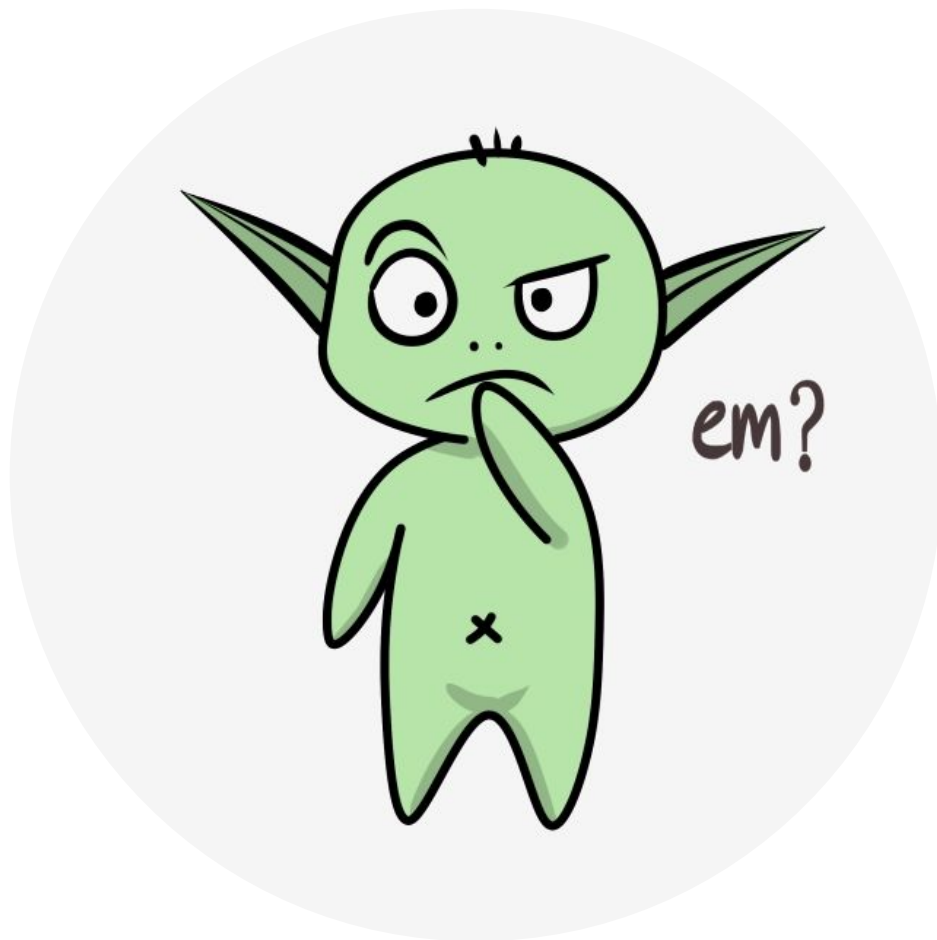
- **Updates Model and Policy via Search**

The learned model is improved using real experience, and the policy is trained using MCTS-based action selection

- **Combines Model-Based and Model-Free Learning**

Uses model-based planning (MCTS) at decision time but still trains a policy network to act without search when needed

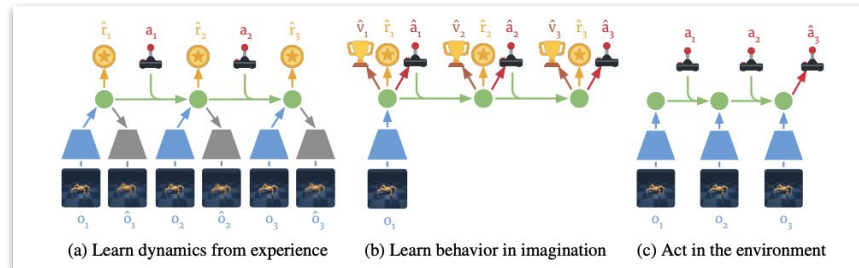




Today: Dreamer

[Hafner et al., 2020; 2021; 2024]

“(...) a reinforcement learning agent that solves long-horizon tasks from images purely by latent imagination.”

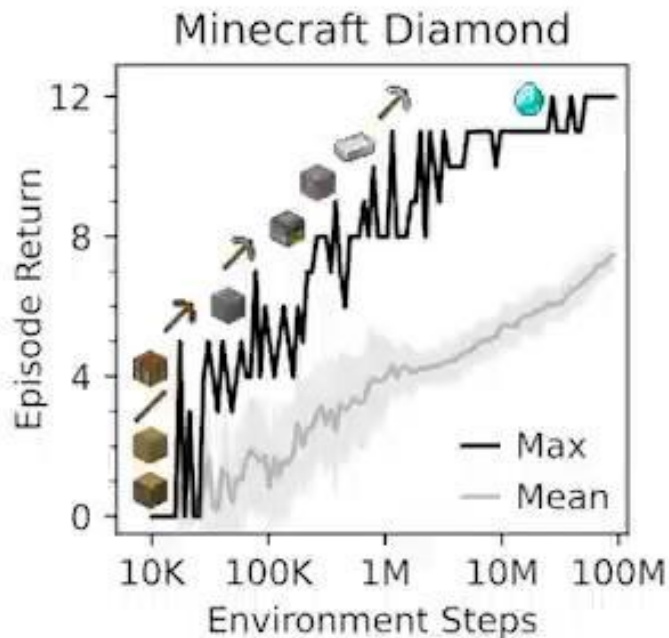


- **Learns a Latent World Model (like MuZero)**
The model predicts latent states, rewards, and values, while also learning to reconstruct the original observations from the latent states
- **Imagines Rollouts in Latent Space (sort of like MuZero)**
The agent generates synthetic trajectories in the learned latent space and uses them for policy training
- **Backpropagates Through the Model**
The gradients from the imagined experience are propagated back through the model, including the reconstruction part
- **Optimizes Policy Using Imagined Experience**
The policy is trained using imagined rollouts and the exact gradients of the learned model

Why Dreamer?



Why Dreamer?



Today: Dreamer v1 [Hafner et al., 2020]

Algorithm 1: Dreamer

Initialize dataset $\mathcal{D}$ with S random seed episodes.
 Initialize neural network parameters θ, ϕ, ψ randomly.
while not converged do

for update step $c = 1..C$ do

// Dynamics learning

Draw B data sequences $\{(a_t, o_t, r_t)\}_{t=k}^{k+L} \sim \mathcal{D}$.
 Compute model states $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$.
 Update θ using representation learning.

// Behavior learning

Imagine trajectories $\{(s_\tau, a_\tau)\}_{\tau=t}^{t+H}$ from each s_t .
 Predict rewards $E(q_\theta(r_\tau | s_\tau))$ and values $v_\psi(s_\tau)$.
 Compute value estimates $V_\lambda(s_\tau)$ via Equation 6.
 Update $\phi \leftarrow \phi + \alpha \nabla_\phi \sum_{\tau=t}^{t+H} V_\lambda(s_\tau)$.

Update $\psi \leftarrow \psi - \alpha \nabla_\psi \sum_{\tau=t}^{t+H} \frac{1}{2} \|v_\psi(s_\tau) - V_\lambda(s_\tau)\|^2$

// Environment interaction

$o_1 \leftarrow \text{env.reset}()$

for time step $t = 1..T$ do

Compute $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$ from history.
 Compute $a_t \sim q_\phi(a_t | s_t)$ with the action model.
 Add exploration noise to action.

$r_t, o_{t+1} \leftarrow \text{env.step}(a_t)$.

Add experience to dataset $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_t, a_t, r_t)_{t=1}^T\}$.

Model components

Representation $p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$

Transition $q_\theta(s_t | s_{t-1}, a_{t-1})$

Reward $q_\theta(r_t | s_t)$

Action $q_\phi(a_t | s_t)$

Value $v_\psi(s_t)$

Hyper parameters

Seed episodes S

Collect interval C

Batch size B

Sequence length L

Imagination horizon H

Learning rate α

“we use p for distributions that generate samples in the real environment and q for their approximations that enable latent imagination”

Dynamics Learning [Hafner et al., 2020]

- Dreamer v1 has different concerns:
 - Reward prediction: predicts the distribution of rewards (allowing for stochasticity)
 - Image reconstruction: used only to provide a learning signal (also a distribution)
 - Mismatch between the posterior distribution (which incorporates observations) and the prior distribution (which is used for imagination)

$$\mathcal{J}_{\text{REC}} \doteq \mathbb{E}_p \left(\sum_t \left(\mathcal{J}_{\text{O}}^t + \mathcal{J}_{\text{R}}^t + \mathcal{J}_{\text{D}}^t \right) \right) + \text{const} \quad \mathcal{J}_{\text{O}}^t \doteq \ln q(o_t \mid s_t)$$

$$\mathcal{J}_{\text{R}}^t \doteq \ln q(r_t \mid s_t) \quad \mathcal{J}_{\text{D}}^t \doteq -\beta \text{KL} \left(p(s_t \mid s_{t-1}, a_{t-1}, o_t) \parallel q(s_t \mid s_{t-1}, a_{t-1}) \right)$$

Behavior Learning [Hafner et al., 2020]

- Consider imagined trajectories with a finite horizon H ; Dreamer goes beyond such a horizon length by using an actor-critic model (in latent space)

Action model: $a_\tau \sim q_\phi(a_\tau \mid s_\tau)$

Value model: $v_\psi(s_\tau) \approx \mathbb{E}_{q(\cdot \mid s_\tau)} \left(\sum_{\tau=t}^{t+H} \gamma^{\tau-t} r_\tau \right)$

- Actions are defined by a tanh, but to see “sampled actions as deterministically dependent on the neural network output, we apply the reparameterization trick”:

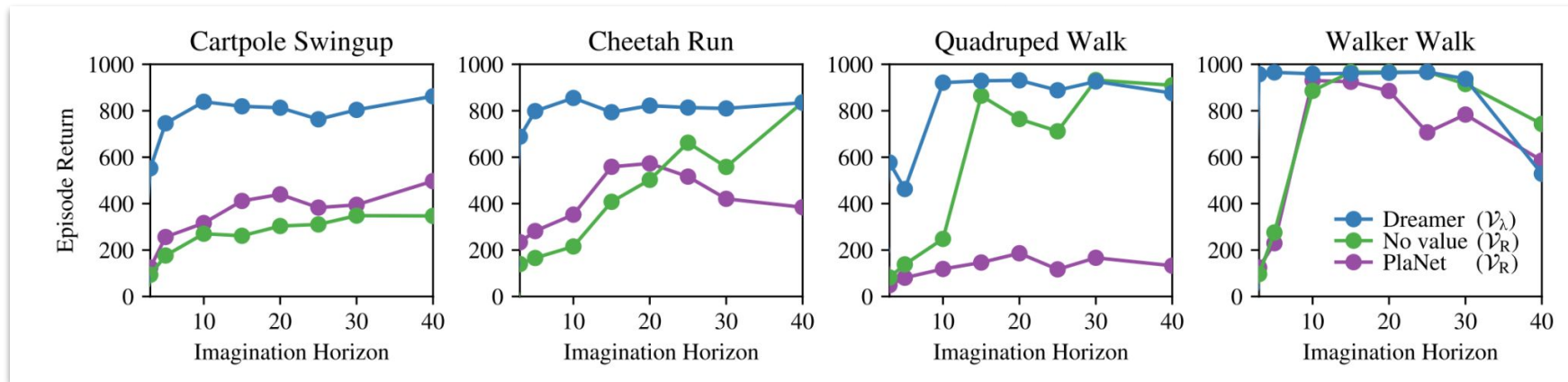
$$a_\tau = \tanh(\mu_\phi(s_\tau) + \sigma_\phi(s_\tau) \epsilon), \quad \epsilon \sim \text{Normal}(0, \mathbb{I})$$

- Dreamer uses a λ -return as target, on imagined transitions in latent space

$$V_\lambda(s_\tau) \doteq (1 - \lambda) \sum_{n=1}^{H-1} \lambda^{n-1} V_N^n(s_\tau) + \lambda^{H-1} V_N^H(s_\tau)$$

Behavior Learning [Hafner et al., 2020]

- Consider imagined trajectories with a finite horizon H ; Dreamer goes beyond such a horizon length by using an actor-critic model (in latent space)



- Dreamer uses a λ -return as target, on imagined transitions in latent space

$$V_\lambda(s_\tau) \doteq (1 - \lambda) \sum_{n=1}^{H-1} \lambda^{n-1} V_N^n(s_\tau) + \lambda^{H-1} V_N^H(s_\tau)$$

Behavior Learning [Hafner et al., 2020]

- As usual, they end up minimizing a MSE loss in the regression task

$$\text{Actor: } \max_{\phi} \mathbb{E}_{q_{\theta}, q_{\phi}} \left(\sum_{\tau=t}^{t+H} V_{\lambda}(s_{\tau}) \right) \quad \text{Critic: } \min_{\psi} \mathbb{E}_{q_{\theta}, q_{\phi}} \left(\sum_{\tau=t}^{t+H} \frac{1}{2} \|v_{\psi}(s_{\tau}) - V_{\lambda}(s_{\tau})\|^2 \right)$$

- “The action model uses analytic gradients through the learned dynamics to maximize the value estimates. To understand this, we note that the value estimates depend on the reward and value predictions, which depend on the imagined states, which in turn depend on the imagined actions. Since all steps are implemented as neural networks, we analytically compute by stochastic backpropagation.”

Dreamer v1 [Hafner et al., 2020]

Algorithm 1: Dreamer

Initialize dataset $\mathcal{D}$ with S random seed episodes.
 Initialize neural network parameters θ, ϕ, ψ randomly.

while not converged do

for update step $c = 1..C$ **do**

 // Dynamics learning

 Draw B data sequences $\{(a_t, o_t, r_t)\}_{t=k}^{k+L} \sim \mathcal{D}$.

 Compute model states $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$.

 Update θ using representation learning.

 // Behavior learning

 Imagine trajectories $\{(s_\tau, a_\tau)\}_{\tau=t}^{t+H}$ from each s_t .

 Predict rewards $E(q_\theta(r_\tau | s_\tau))$ and values $v_\psi(s_\tau)$.

 Compute value estimates $V_\lambda(s_\tau)$ via Equation 6.

 Update $\phi \leftarrow \phi + \alpha \nabla_\phi \sum_{\tau=t}^{t+H} V_\lambda(s_\tau)$.

 Update $\psi \leftarrow \psi - \alpha \nabla_\psi \sum_{\tau=t}^{t+H} \frac{1}{2} \|v_\psi(s_\tau) - V_\lambda(s_\tau)\|^2$.

 // Environment interaction

$o_1 \leftarrow \text{env.reset}()$

for time step $t = 1..T$ **do**

 Compute $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$ from history.

 Compute $a_t \sim q_\phi(a_t | s_t)$ with the action model.

 Add exploration noise to action.

$r_t, o_{t+1} \leftarrow \text{env.step}(a_t)$.

 Add experience to dataset $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_t, a_t, r_t)_{t=1}^T\}$.

Model components

Representation $p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$

Transition $q_\theta(s_t | s_{t-1}, a_{t-1})$

Reward $q_\theta(r_t | s_t)$

Action $q_\phi(a_t | s_t)$

Value $v_\psi(s_t)$

Hyper parameters

Seed episodes S

Collect interval C

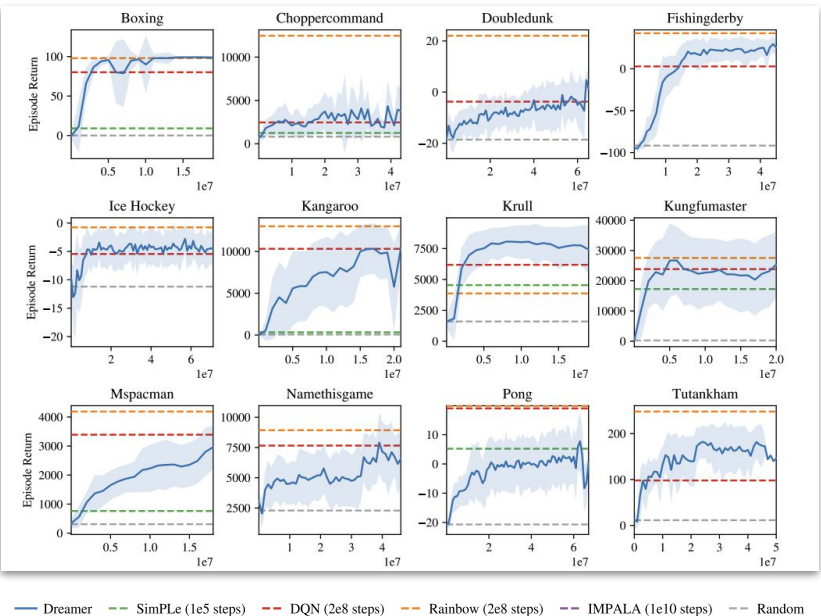
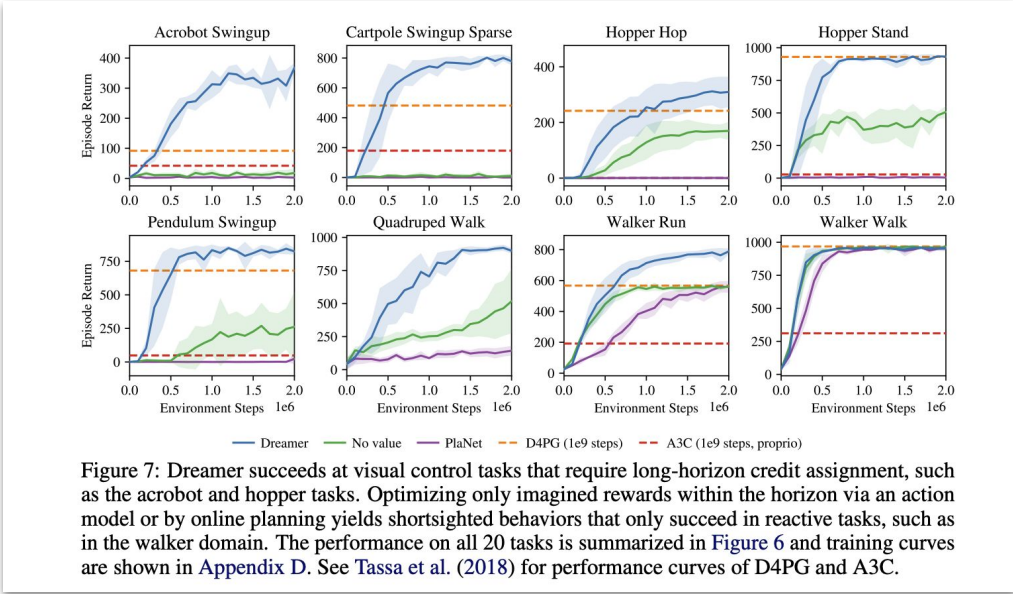
Batch size B

Sequence length L

Imagination horizon H

Learning rate α

Dreamer v1 [Hafner et al., 2020]





Dreamer v2 [Hafner et al., 2021]

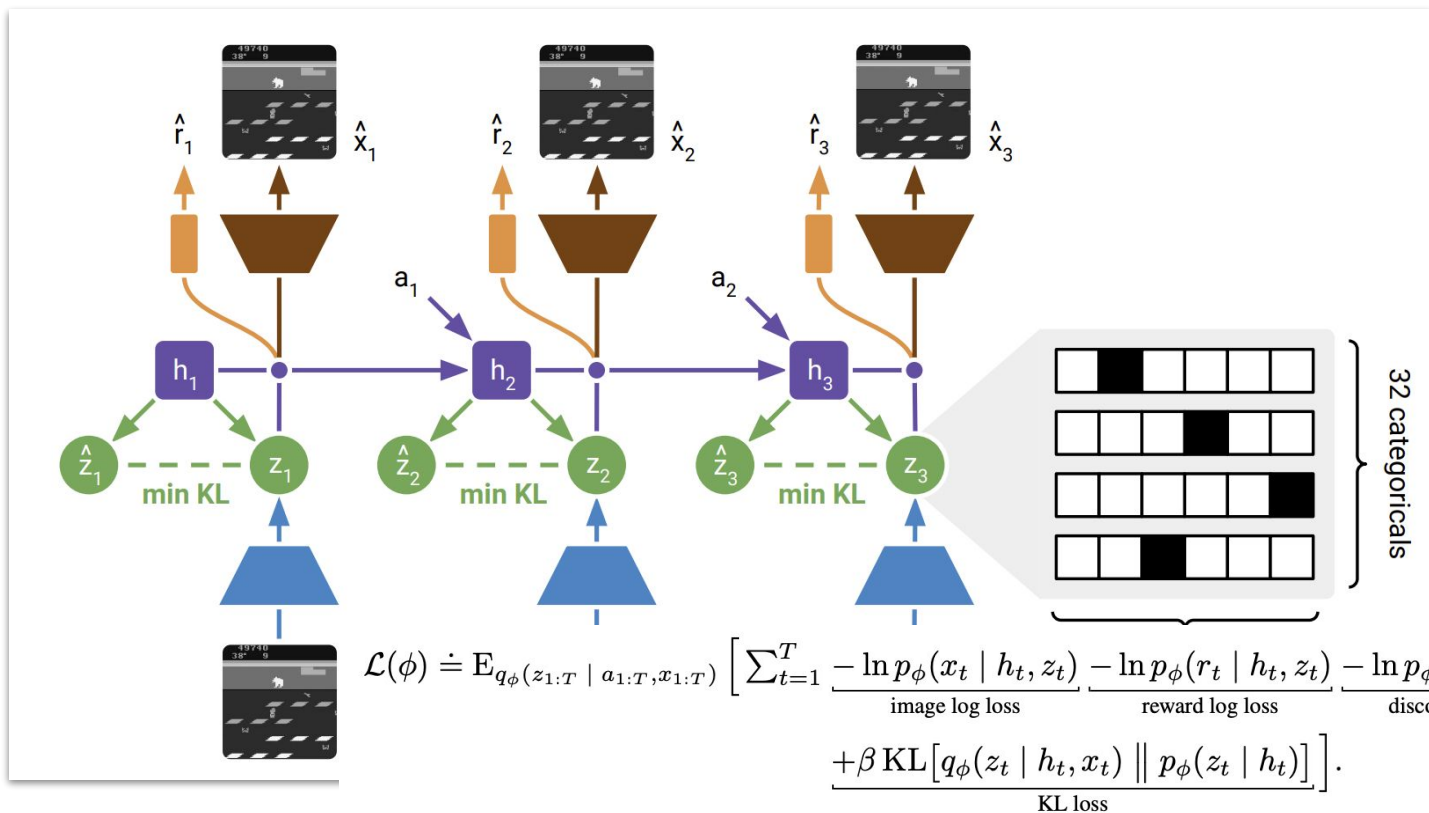
- “While learning world models from image inputs has recently become feasible for some tasks, modeling Atari games accurately enough to derive successful behaviors has remained an open challenge for many years”
- the “MuZero agent (Schrittwieser et al., 2019) shows that planning can achieve impressive performance on board games and deterministic Atari games given extensive engineering effort and a vast computational budget. However, its implementation is not available to the public and it would require over 2 months of computation to train even one agent on a GPU, rendering it impractical for most research groups.”

Dreamer v2 [Hafner et al., 2021]

Summary of changes that were tried and were found to help:

- **Categorical latents** Using categorical latent states using straight-through gradients in the world model instead of Gaussian latents with reparameterized gradients.
- **KL balancing** Separately scaling the prior cross entropy and the posterior entropy in the KL loss to encourage learning an accurate temporal prior, instead of using free nats.
- **Reinforce only** Reinforce gradients worked substantially better for Atari than dynamics backpropagation. For continuous control, dynamics backpropagation worked substantially better.
- **Model size** Increasing the number of units or feature maps per layer of all model components, resulting in a change from 13M parameters to 22M parameters.
- **Policy entropy** Regularizing the policy entropy for exploration both in imagination and during data collection, instead of using external action noise during data collection.

Dreamer v2 [Hafner et al., 2021]



Dreamer v2 – Loss Functions [Hafner et al., 2021]

- The *world model*:

$$\mathcal{L}(\phi) \doteq \mathbb{E}_{q_\phi(z_{1:T} \mid a_{1:T}, x_{1:T})} \left[\underbrace{\sum_{t=1}^T \frac{-\ln p_\phi(x_t \mid h_t, z_t)}{\text{image log loss}}}_{\text{image log loss}} \underbrace{\frac{-\ln p_\phi(r_t \mid h_t, z_t)}{\text{reward log loss}}}_{\text{reward log loss}} \underbrace{\frac{-\ln p_\phi(\gamma_t \mid h_t, z_t)}{\text{discount log loss}}}_{\text{discount log loss}} \right. \\ \left. \underbrace{+ \beta \text{KL}[q_\phi(z_t \mid h_t, x_t) \parallel p_\phi(z_t \mid h_t)]}_{\text{KL loss}} \right]$$

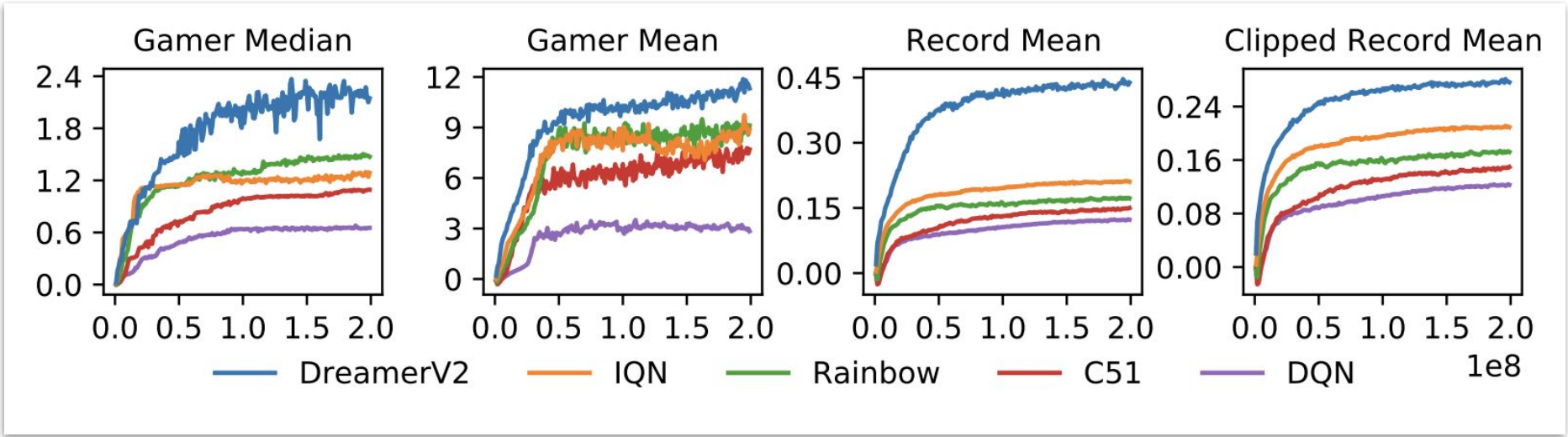
- Critic*:

$$\mathcal{L}(\xi) \doteq \mathbb{E}_{p_\phi, p_\psi} \left[\sum_{t=1}^{H-1} \frac{1}{2} (v_\xi(\hat{z}_t) - \text{sg}(V_t^\lambda))^2 \right]$$

- Actor*:

$$\mathcal{L}(\psi) \doteq \mathbb{E}_{p_\phi, p_\psi} \left[\sum_{t=1}^{H-1} \left(\underbrace{-\rho \ln p_\psi(\hat{a}_t \mid \hat{z}_t) \text{sg}(V_t^\lambda - v_\xi(\hat{z}_t))}_{\text{reinforce}} \underbrace{-(1 - \rho)V_t^\lambda}_{\text{dynamics backprop}} \underbrace{-\eta \text{H}[a_t \mid \hat{z}_t]}_{\text{entropy regularizer}} \right) \right]$$

Dreamer v2 [Hafner et al., 2021]



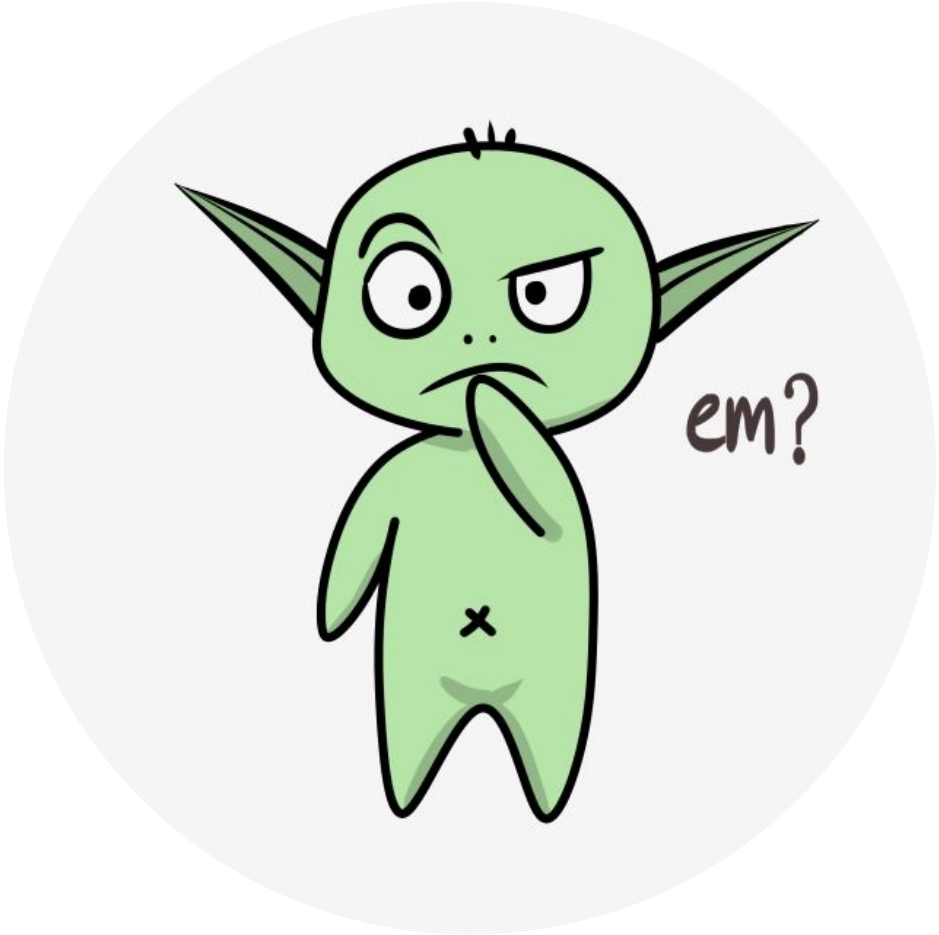
Dreamer v2 – Ablations [Hafner et al., 2021]

Agent	Gamer Median	Gamer Mean	Record Mean	Clipped Record Mean
DreamerV2	1.64	11.33	0.36	0.25
No Layer Norm	1.66	5.95	0.38	0.25
No Reward Gradients	1.68	6.18	0.37	0.24
No Discrete Latents	1.08	3.71	0.24	0.19
No KL Balancing	0.84	3.49	0.19	0.16
No Policy Reinforce	0.69	2.74	0.16	0.15
No Image Gradients	0.04	0.31	0.01	0.01

Table 2: Ablations to DreamerV2 measured by their Atari performance at 200M frames, sorted by the last column. This experiment uses a slightly earlier version of DreamerV2 compared to [Table 1](#). Each ablation only removes one part of the DreamerV2 agent. Discrete latent variables and KL balancing substantially contribute to the success of DreamerV2. Moreover, the world model relies on image gradients to learn general representations that lead to successful behaviors, even if the representations are not specifically learned for predicting past rewards.

Dreamer v2 – Big Picture View [Hafner et al., 2021]

Algorithm	Reward Modeling	Image Modeling	Latent Transitions	Single GPU	Trainable Parameters	Atari Frames	Accelerator Days
DreamerV2	✓	✓	✓	✓	22M	200M	10
SimPLe	✓	✓	✗	✓	74M	4M	40
MuZero	✓	✗	✓	✗	40M	20B	80
MuZero Reanalyze	✓	✗	✓	✗	40M	200M	80



Dreamer v3 [Hafner et al., 2023]

- “We present DreamerV3, a general algorithm that outperforms specialized methods across over 150 diverse tasks, with a single configuration”
- “Robustness techniques based on normalization, balancing, and transformations enable stable learning across domains”
- “Creating a general algorithm that learns to master new domains without having to be reconfigured has been a central challenge in artificial intelligence and would open up reinforcement learning to a wide range of practical applications”

Dreamer v3 [Hafner et al., 2023]

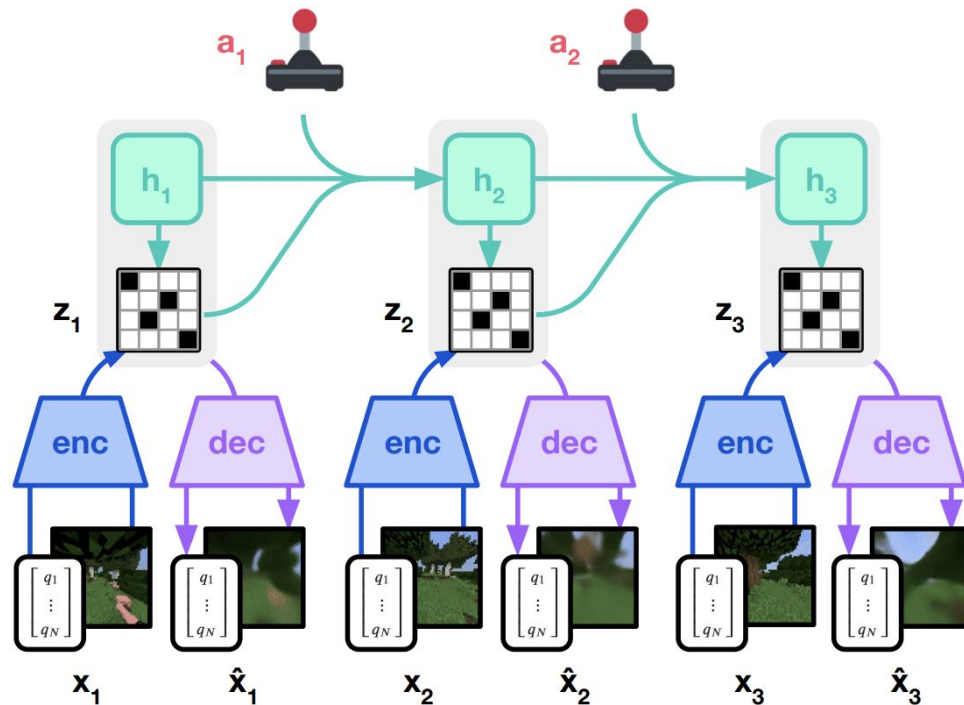
We summarize the changes that DreamerV3 introduces as follows:

- **Robustness techniques:** Observation symlog, KL balance and free bits, 1% unimix for all categorical, percentile return normalization, symexp twohot loss for the reward head and critic.
- **Network architecture:** Block GRU, RMSNorm normalization, SiLu activation.
- **Optimizer:** Adaptive gradient clipping (AGC), LaProp (RMSProp followed by momentum).
- **Replay buffer:** Larger capacity, online queue, storing and updating latent states.

Dreamer v3 – World Model Learning [Hafner et al., 2023]

- Uses an autoencoder, but it is still implemented as a Recurrent State-Space Model

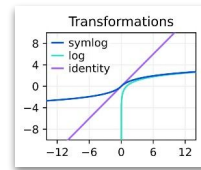
$$\mathcal{L}(\phi) \doteq \mathbb{E}_{q_\phi} \left[\sum_{t=1}^T (\beta_{\text{pred}} \mathcal{L}_{\text{pred}}(\phi) + \beta_{\text{dyn}} \mathcal{L}_{\text{dyn}}(\phi) + \beta_{\text{rep}} \mathcal{L}_{\text{rep}}(\phi)) \right]$$



The symlog function and the symexp twohot loss [Hafner et al., 2023]

- “The symlog function compresses the magnitudes of both large positive and negative values. Unlike the logarithm, it is symmetric around the origin while preserving the input sign. This allows the optimization process to quickly move the network predictions to large values when needed”

$$\text{symlog}(x) \doteq \text{sign}(x) \ln(|x| + 1) \quad \text{symexp}(x) \doteq \text{sign}(x) (\exp(|x|) - 1)$$



- The symexp twohot loss deals with potentially stochastic targets: the network outputs the logits for a softmax distribution over exponentially spaced bins; and they compute the weighted sum to generate the output

$$\hat{y} \doteq \text{softmax}(f(x))^T B \quad B \doteq \text{symexp}([-20 \quad \dots \quad +20])$$

Dreamer v3 – World Model Learning [Hafner et al., 2023]

$$\mathcal{L}_{\text{pred}}(\phi) \doteq -\ln p_{\phi}(x_t \mid z_t, h_t) - \ln p_{\phi}(r_t \mid z_t, h_t) - \ln p_{\phi}(c_t \mid z_t, h_t)$$

$$\mathcal{L}_{\text{dyn}}(\phi) \doteq \max(1, \text{KL}[\text{sg}(q_{\phi}(z_t \mid h_t, x_t)) \parallel p_{\phi}(z_t \mid h_t)])$$

$$\mathcal{L}_{\text{rep}}(\phi) \doteq \max(1, \text{KL}[q_{\phi}(z_t \mid h_t, x_t) \parallel \text{sg}(p_{\phi}(z_t \mid h_t))])$$

- “ $\mathcal{L}_{\text{pred}}$ trains the decoder and reward predictor via the symlog squared loss (...), and the continue predictor via logistic regression”
 - “ $\mathcal{L}_{\text{dyn}}$ trains the sequence model to predict the next representation by minimizing the KL divergence between the predictor and the next stochastic representation”
 - “ $\mathcal{L}_{\text{rep}}$ trains the representations to become more predictable allowing us to use a factorized dynamics predictor for fast sampling during imagination training”
- “we employ free bits by clipping the dynamics and representation losses”

Dreamer v3 – Critic Learning [Hafner et al., 2023]

- “The actor and critic neural networks learn behaviors purely from abstract trajectories of representations predicted by the world model”
- “we parameterize the critic as categorical distribution with exponentially spaced bins, decoupling the scale of gradients from the prediction targets”
- “we stabilize learning by regularizing the critic towards predicting the outputs of an exponentially moving average of its own parameters. This is similar to target networks used previously in reinforcement learning but allows us to compute returns using the current critic network”

$$\mathcal{L}(\psi) \doteq - \sum_{t=1}^T \ln p_{\psi}(R_t^{\lambda} \mid s_t) \quad R_t^{\lambda} \doteq r_t + \gamma c_t \left((1 - \lambda)v_t + \lambda R_{t+1}^{\lambda} \right) \quad R_T^{\lambda} \doteq v_T$$

Dreamer v3 – Actor Learning [Hafner et al., 2023]

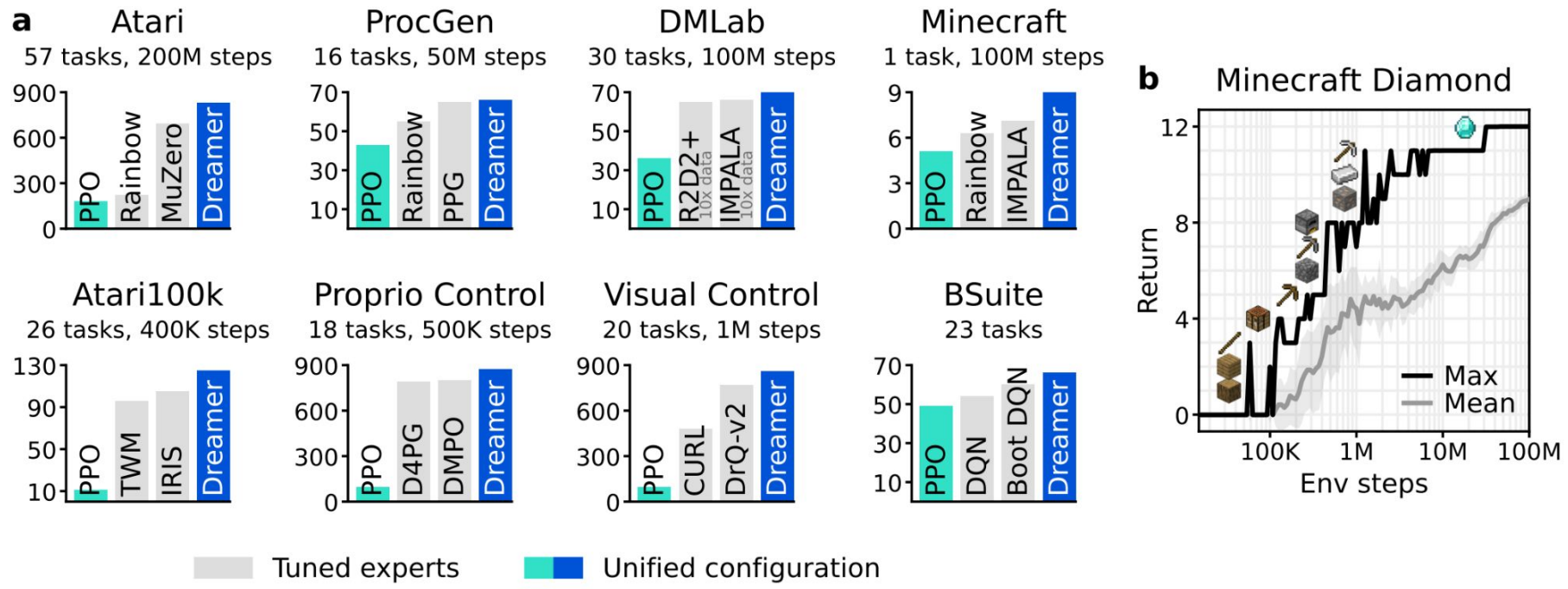
- Uses entropy regularization, and to have a fixed coefficient they normalize returns to be approximately contained in the interval $[0, 1]$
- They are still using Reinforce

$$\mathcal{L}(\theta) \doteq - \sum_{t=1}^T \text{sg}\left((R_t^\lambda - v_\psi(s_t)) / \max(1, S)\right) \log \pi_\theta(a_t \mid s_t) + \eta H[\pi_\theta(a_t \mid s_t)]$$

$$S \doteq \text{EMA}(\text{Per}(R_t^\lambda, 95) - \text{Per}(R_t^\lambda, 5), 0.99)$$

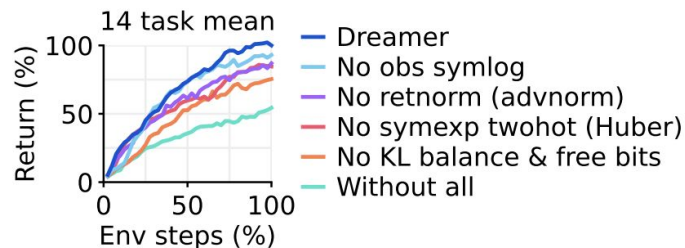
... but they do a lot of normalization

Dreamer v3 [Hafner et al., 2023]

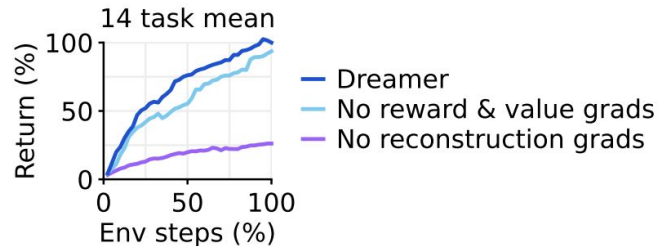


Dreamer v3 – Ablations and Robust Scaling [Hafner et al., 2023]

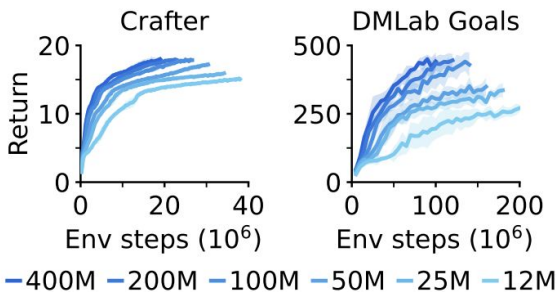
a Robustness techniques



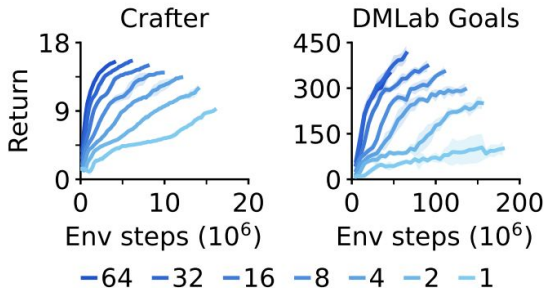
b Learning signals



c Model size scaling



d Replay scaling



Next class

- Midterm!
- Then paper seminars
- Assignment 4 is due on March 28
- Paper commentary is due on April 9

